

CHƯƠNG 7. NGUYÊN TẮC THIẾT KẾ

Áp dụng cho hệ thống Pet Service

Trong chương này, nhóm em đánh giá cách tổ chức code của hệ thống Pet Service dựa trên hai khái niệm Coupling, Cohesion và năm nguyên lý SOLID. Phần đánh giá được viết dựa trên code hiện tại của dự án, bao gồm cả phần backend vừa được tái cấu trúc sang dạng module. Vì vậy, các ví dụ bên dưới đều lấy trực tiếp từ cấu trúc và các file đang có trong thư mục Pet_Service.

7.1. Áp dụng Design Concepts

7.1.1. Giới thiệu chung về Coupling và Cohesion

Cohesion có thể hiểu là mức độ liên quan giữa các phần code nằm trong cùng một file hoặc module. Nếu một file chỉ tập trung giải quyết một nhóm công việc cụ thể thì file đó có cohesion cao. Ngược lại, nếu một file vừa gọi database, vừa xử lý nghiệp vụ, vừa format dữ liệu và còn quản lý giao diện thì cohesion sẽ thấp và khó bảo trì hơn.

Coupling là mức độ phụ thuộc giữa các file hoặc module với nhau. Coupling thấp không có nghĩa là các phần hoàn toàn không liên quan, mà là chúng liên hệ thông qua những đầu mối rõ ràng. Ví dụ, component gọi service thay vì gọi database trực tiếp. Khi API thay đổi thì chủ yếu sửa ở service, không cần sửa toàn bộ giao diện.

7.1.2. Đánh giá phía Frontend

7.1.2.1. Cohesion phía Frontend

Ưu điểm:

- Frontend đã được chia theo từng nhóm người dùng gồm admin, customer, doctor, staff và auth. Mỗi nhóm có page và service riêng nên khá dễ tìm code khi cần sửa một chức năng.
- Các component giao diện cũng được chia theo nghiệp vụ. Ví dụ phần customer có appointments, boarding, history, medications, notifications, pets và profile. Cách chia này phù hợp với các chức năng đã mô tả trong báo cáo ITSSfinal2.
- Phần gọi API của khách hàng được tách thành các file như customerAppointmentsApi.ts, customerBoardingApi.ts, customerPetsApi.ts. Component không cần biết câu truy vấn database phía sau được viết như thế nào.
- Dự án có thư mục components/ui chứa Button, Card, Dialog, Table, Input và nhiều component dùng chung. Điều này giúp hạn chế việc mỗi màn hình tự viết lại một bộ giao diện giống nhau.

Nhược điểm:

- Một số file vẫn còn khá dài và làm nhiều việc cùng lúc. Ví dụ StaffBoardingTab.tsx vừa hiển thị danh sách lưu trú, quản lý phòng, cập nhật chăm sóc, checkout và mở nhiều modal. CustomerAppointmentModals.tsx cũng gom nhiều loại popup đặt lịch, đổi lịch và lưu trú trong cùng một file.
- CustomerPortal.tsx và một số màn hình admin còn giữ nhiều state, xử lý load dữ liệu, chuyển tab và điều khiển modal. Khi thêm chức năng mới thì các file này sẽ tiếp tục dài ra.
- Cách gọi API chưa hoàn toàn thống nhất. Nhóm customer dùng requestJson.ts nhưng doctor và staff lại có fetchWithAuth riêng. Một vài component chuyên khoa còn gọi fetch trực tiếp.

Đề xuất cải thiện:

- Tách các file lớn thành component nhỏ và custom hook. Chẳng hạn StaffBoardingTab có thể tách thành danh sách khách lưu trú, quản lý phòng, form checkout và hook xử lý dữ liệu.
- Dùng chung một hàm gọi API cho cả customer, doctor, staff và admin để cách gán token, đọc lỗi và parse JSON giống nhau.
- Tách phần type ra khỏi các service quá dài để file service chỉ còn các hàm gọi API.

7.1.2.2. Coupling phía Frontend**Ưu điểm:**

- Frontend không kết nối Supabase trực tiếp mà đi qua Express API. Đây là điểm tốt vì giao diện không bị phụ thuộc vào bảng và cột trong database.
- apiUrl.ts xử lý địa chỉ backend, authSession.ts xử lý token và requestJson.ts xử lý request JSON. Các phần dùng chung này giúp giảm code lặp.
- TypeScript interface được dùng khá nhiều trong các luồng đặt lịch, thuốc, thông báo và thú cưng. Nhờ đó component biết rõ dữ liệu nhận được gồm những trường nào.

Nhược điểm:

- URL API vẫn được viết trực tiếp trong từng service. Nếu đổi toàn bộ prefix hoặc format response thì phải sửa ở nhiều nơi.
- Backend đang trả về một số class màu của Tailwind như cls và dot cho trạng thái lịch khám. Cách này tiện cho frontend nhưng làm backend bị phụ thuộc vào cách trang web hiển thị.
- App.tsx dùng nhiều câu lệnh if theo role để chọn portal. Nếu sau này có thêm vai trò mới thì vẫn phải sửa file trung tâm này.

Đề xuất cải thiện:

- Backend chỉ nên trả status và label; màu sắc hoặc class CSS nên xử lý trong frontend.

- Có thể tạo cấu hình ánh xạ role với portal thay vì viết nhiều câu lệnh if.
- Chuẩn hóa response theo một dạng chung như { ok, data, message } để service dễ dùng và dễ test hơn.

7.1.3. Đánh giá phía Backend

7.1.3.1. Cohesion phía Backend

Ưu điểm:

- Sau khi tái cấu trúc, app.js và server.js đã được tách riêng. app.js tạo Express app và gắn route, còn server.js chỉ khởi động server và lịch gửi email nhắc nhở. Cách tách này rõ ràng hơn cấu trúc cũ.
- Backend được chia theo module nghiệp vụ như auth, customers, pets, appointments, staff, doctors, medical, boarding, billing và notifications. Khi cần tìm chức năng khám bệnh hoặc lưu trú có thể đi thẳng vào module tương ứng.
- Phần cấu hình, middleware và thư viện chung đã có thư mục riêng. auth.middleware.js xử lý xác thực và kiểm tra role nên route không phải viết lại phần này.
- modules/index.js gom các route và prefix API vào một chỗ. app.js chỉ cần duyệt danh sách module để gắn vào ứng dụng.

Nhược điểm:

- medical.routes.js vẫn gần 2.000 dòng. File này không chỉ khai báo route mà còn truy vấn Supabase, xử lý kết quả khám, đơn thuốc, dịch vụ chuyên khoa và thông báo. Vì vậy route chưa thật sự mỏng.
- admin.service.js và staff.service.js đang chứa quá nhiều nhóm chức năng. admin.service vừa xử lý người dùng, dịch vụ, lịch hẹn, nhân sự, báo cáo và cài đặt. staff.service vừa xử lý lịch hẹn, grooming, lưu trú và thanh toán.
- Module reports và grooming đã có thư mục riêng nhưng hiện tại chủ yếu lấy lại hàm từ admin.service và staff.service. Như vậy mới tách tên module chứ logic chưa chuyển hẳn về đúng module.
- Một số route vẫn gọi Supabase trực tiếp. Điều này làm route vừa nhận request vừa biết chi tiết database.

Đề xuất cải thiện:

- Tách medical.routes.js thành route, controller và các service nhỏ như khám bệnh, đơn thuốc, dịch vụ chuyên khoa và thông báo bác sĩ.
- Tách admin.service.js theo từng phần dashboard, user, service, report và settings. Staff service cũng nên tách appointment, grooming, boarding và payment.
- Chuyển logic của reports và grooming về đúng module của nó thay vì chỉ export lại từ module khác.

7.1.3.2. Coupling phía Backend

Ưu điểm:

- Route được gắn qua apiModules nên app.js không cần biết chi tiết từng endpoint.
- JWT, Prisma và Supabase client được đặt trong lib. Các file khác dùng lại client có sẵn thay vì tự tạo kết nối.
- Middleware xác thực được dùng chung cho customer, doctor, staff và admin. Việc phân quyền vì vậy không bị viết rải rác ở từng handler.

Nhược điểm:

- Phần lớn service vẫn import Supabase trực tiếp và gọi tên bảng, tên cột. Nếu đổi cách truy cập database thì phải sửa nhiều service.
- Các service gọi lẫn nhau khá nhiều, ví dụ appointment và boarding gọi email service, admin gọi doctor schedule service. Các phụ thuộc này hiện vẫn là import trực tiếp.
- Biến môi trường chưa được gom hết vào config/env.js. SMTP, thông tin phòng khám, timezone và storage bucket vẫn được đọc ở một số service khác nhau.

Đề xuất cải thiện:

- Tạo lớp repository cho các phần appointment, pet, invoice và boarding. Service xử lý nghiệp vụ, repository xử lý Supabase.
- Gom toàn bộ biến môi trường vào một module config và kiểm tra dữ liệu cấu hình ngay khi server khởi động.
- Với email và notification, có thể tách thành bước xử lý sau khi nghiệp vụ chính hoàn thành để service không gọi SMTP trực tiếp.

7.1.4. Kết luận

Nhìn chung, cấu trúc hiện tại của Pet Service đã tốt hơn sau khi tái cấu trúc. Frontend chia theo role và nghiệp vụ, backend chia theo module và đã tách app khỏi server. Tuy nhiên, bên trong một số module vẫn còn các file quá lớn nên việc chia thư mục chưa giải quyết hết vấn đề. Phần cần làm tiếp theo là tách nhỏ các file này, thống nhất cách gọi API và đưa phần truy cập database ra khỏi route.

7.2. Áp dụng Design Principles SOLID

7.2.1. Giới thiệu chung

SOLID gồm năm nguyên lý thường được dùng để tổ chức code sao cho dễ đọc, dễ sửa và dễ mở rộng. Dự án Pet Service không xây dựng nhiều class kế thừa như ví dụ hướng đối tượng truyền thống, nhưng các nguyên lý này vẫn có thể áp dụng cho component, service, module và các kiểu dữ liệu TypeScript.

- S - Single Responsibility Principle: một file hoặc module nên tập trung vào một trách nhiệm chính.
- O - Open/Closed Principle: có thể mở rộng chức năng mà hạn chế sửa phần code cũ đã ổn định.
- L - Liskov Substitution Principle: các phần cùng một contract phải có thể thay thế cho nhau mà không làm caller bị lỗi.
- I - Interface Segregation Principle: nên tách interface hoặc service lớn thành các phần nhỏ đúng với nhu cầu sử dụng.
- D - Dependency Inversion Principle: logic nghiệp vụ không nên phụ thuộc trực tiếp vào chi tiết database, email hoặc storage.

7.2.2. Đánh giá việc áp dụng SOLID

7.2.2.1. Single Responsibility Principle (SRP)

Phần đã áp dụng:

- app.js và server.js đã có nhiệm vụ riêng. Đây là ví dụ rõ nhất cho SRP sau khi backend được tái cấu trúc.
- Các module auth, pets, appointments, boarding và notifications được đặt riêng theo nghiệp vụ.
- Frontend có service, type và component riêng cho nhiều chức năng của customer.

Phần chưa tốt:

- medical.routes.js, admin.service.js và staff.service.js vẫn có quá nhiều lý do để thay đổi.
- Một số component lớn vừa giữ state, gọi API, validate form và render nhiều modal.

Cách cải thiện:

- Tách service theo từng use case và giữ route chỉ làm nhiệm vụ nhận request, gọi service rồi trả response.
- Tách component lớn thành container, component hiển thị và custom hook.

7.2.2.2. Open/Closed Principle (OCP)

Phần đã áp dụng:

- Danh sách apiModules cho phép thêm một route group mới mà không cần viết thêm handler trong app.js.
- Frontend có thể thêm page hoặc component mới trong từng feature mà không phải sửa các component UI dùng chung.

Phần chưa tốt:

- Khi thêm role mới vẫn phải sửa App.tsx.

- Khi có trạng thái hoặc luồng xử lý mới, nhiều đoạn switch, if hoặc object mapping cũ vẫn phải sửa trực tiếp.

Cách cải thiện:

- Tạo cấu hình role - portal và registry cho các handler theo trạng thái.
- Mỗi module chỉ export public API; logic mới nên được thêm trong module sở hữu nghiệp vụ đó.

7.2.2.3. Liskov Substitution Principle (LSP)

Trong code hiện tại chưa có hệ thống class cha - class con cho các dịch vụ nên không thể đánh giá LSP theo cách HotelService kế thừa Service như ví dụ lý thuyết. Với dự án này, LSP phù hợp hơn khi xét các implementation cùng một contract.

Điểm hiện tại:

- TypeScript đã mô tả khá rõ kiểu dữ liệu mà service trả về.
- Tuy nhiên service backend đang dùng Supabase trực tiếp, chưa có interface repository để thay bằng Prisma hoặc dữ liệu giả khi test.

Cách cải thiện:

- Định nghĩa contract cho repository và giữ cùng kiểu dữ liệu, cách báo lỗi giữa các implementation.
- Viết test dùng chung cho repository thật và repository giả để kiểm tra khả năng thay thế.

7.2.2.4. Interface Segregation Principle (ISP)

Phần đã áp dụng:

- Các API của customer được chia nhỏ theo appointment, boarding, medication, notification và pet.
- Component thường chỉ nhận các props cần dùng thay vì nhận toàn bộ dữ liệu của portal.

Phần chưa tốt:

- staffAppointmentsService đang chứa cả profile, lịch hẹn, grooming, boarding, payment và walk-in.
- doctorAppointmentsService chứa thông báo, lịch khám, khám bệnh và dịch vụ chuyên khoa trong cùng một object.
- admin.service.js ở backend cũng là một service quá rộng.

Cách cải thiện:

- Tách service theo đúng nhóm chức năng mà caller cần, ví dụ BoardingApi, PaymentApi, DoctorExamApi và DoctorNotificationApi.
- Không export các helper nội bộ nếu module khác không cần sử dụng.

7.2.2.5. Dependency Inversion Principle (DIP)

Phần đã áp dụng:

- Component frontend gọi service thay vì truy cập database.
- Route backend phần lớn gọi service thay vì để frontend quyết định nghiệp vụ.
- Supabase client, Prisma client và JWT helper đã được gom trong thư mục lib.

Phần chưa tốt:

- Business service vẫn phụ thuộc trực tiếp vào Supabase SDK.
- Email, PDF, storage và process.env vẫn được gọi trực tiếp trong một số service.

Cách cải thiện:

- Tạo các abstraction đơn giản như AppointmentRepository, EmailSender, FileStorage và PdfGenerator.
- Truyền implementation thật khi khởi tạo ứng dụng và dùng implementation giả khi unit test.

7.2.3. Tổng hợp và hướng cải thiện

Nguyên lý	Mức độ	Nhận xét ngắn
SRP	Khá	Cấu trúc module rõ nhưng còn nhiều file lớn.
OCP	Khá	Có thể thêm module, nhưng role và status còn hard-code.
LSP	Trung bình	Có type nhưng chưa có repository thay thế được.
ISP	Khá	Customer API tách tốt, doctor/staff/admin còn rộng.
DIP	Trung bình	Có service layer nhưng vẫn phụ thuộc trực tiếp Supabase.

Các việc nên ưu tiên trước:

- Tách medical.routes.js và các service lớn của admin, staff.
- Thống nhất một HTTP client cho toàn bộ frontend.
- Tách component quá dài thành hook, view và modal nhỏ.
- Tạo repository cho appointment, pet, boarding và invoice.
- Đưa màu sắc, class CSS ra khỏi response backend.

7.2.4. Kết luận

Pet Service đã có nhiều phần áp dụng đúng tinh thần SOLID, đặc biệt là việc tách frontend và backend, chia code theo role, chia backend theo module và dùng service layer. Phần tái cấu trúc backend hiện tại là một bước cải thiện rõ vì app, server, config, middleware và các module đã có vị trí riêng.

Tuy vậy, dự án vẫn còn một số file lớn và phụ thuộc trực tiếp vào Supabase. Vì vậy nhóm em đánh giá hệ thống mới áp dụng SOLID ở mức khá, chưa hoàn toàn triệt để. Nếu tiếp tục tách service theo use case, thống nhất contract dữ liệu và thêm repository layer thì code sẽ dễ bảo trì, test và mở rộng hơn mà không cần viết lại toàn bộ hệ thống.

Lưu ý: phần đánh giá này sử dụng code hiện tại trong thư mục Pet_Service, bao gồm phần backend đã tái cấu trúc nhưng chưa commit.